

Bounding Volume Hierarchy (BVH) Report

Implementation summary

The BVH is implemented in the class SceneBVH and built over all scene surfaces using axis-aligned bounding boxes (AABB), stores a minimum and maximum corner in world space.

Each surface is represented by a helper object called SurfaceReference, which stores a pointer to the surface, the surface's world-space AABB and the center of that AABB. This allows the BVH to operate purely on AABBs and indices without duplicating geometry.

Each BVH node is represented by the BVHNode object, which contains a AABB enclosing all surfaces in the node, indices of its left and right child nodes and a range into the surface index array.

Build:

The BVH is built by calling the interface method SceneBVH::build(const Scene& scene).

All scene surfaces are converted into SurfaceReference objects with the AABB queried from each surface. The method buildBvhNode(begin, end) builds the BVH recursively. The node AABB is computed by uniting the bounding volumes of all referenced surfaces. A centroid AABB is built from the surface centers. If the number of surfaces in the range is less than or equal to LEAF_SIZE, a leaf node is created and stores the surface index range directly. If the node is not a leaf, a split is chosen using the SAHSplitter. For each axis (x, y, z), the center AABB extent is divided into a fixed number of bins. Surfaces are assigned to bins based on their bounding box center. For each possible split between adjacent bins, left and right bounding volumes and surface counts are accumulated, which helps for fast iteration over the possible splits. Then the costs are computed and the split with the lowest cost is selected.

$$\text{cost} = \text{leftSurfaceArea} * \text{leftSurfaceCount} + \text{rightSurfaceArea} * \text{rightSurfaceCount}$$

Using std::partition, the surface index range is split into left and right groups according to the selected split plane. If either side would be empty, the node is converted into a leaf to avoid endless recursion. Otherwise, child nodes are built recursively and linked to the parent.

The final BVH is a binary tree stored in a array with node relationships through indices. This layout allows efficient, stack-based traversal during ray intersection and shadow testing.

Traversal details

BVH traversal is implemented iteratively in the SceneBVH class using an stack. Two traversal methods, for finding the closest intersection (intersect) and for shadow testing (occluded).

Traversal for Closest Intersection:

Traversal starts at the root node (index 0). A stack is used to store BVH node indices that still need to be processed. The current valid ray interval is defined by a minimum distance along the ray and a maximum distance initialized to the distance of the closest hit found so far. For each visited node, the ray is tested against the node's bounding box using `AABB::intersect`. If the ray misses the bounding box, the entire subtree below that node is skipped. If the node is a leaf, only the surfaces stored in that node are tested for intersection. Each surface intersection test uses the current closest hit distance as an upper bound, allowing early rejection of farer intersections. When a closer intersection is found the Hit is updated. If the node is an inner node, its left and right child indices are pushed on top of the stack for further traversal. Traversal continues until the stack is empty. The method returns whether any intersection was found and updates the Hit structure with the closest intersection.

Traversal for Shadow Rays:

This traversal follows the same structure as the closest-hit traversal, with the difference that the ray distance interval is limited to the distance to the light source. For leaf nodes, each surface is tested only until the maximum allowed distance. The traversal terminates when any valid intersection is found within this range

Performance comparison

Measured rendering times:

example7	without BVH	449.653 s
example7	with BVH	55.4063 s
example8	without BVH	379.047 s
example8	with BVH	357.021 s

Analysis

example7:

Speedup = $449.653 / 55.4063 = \times 8.12$

Time reduction = $(449.653 - 55.4063) / 449.653 = 87.7\%$ schneller

There are many triangles in the scene, so without a BVH each ray would have to be tested against a very large number of primitives. The BVH quickly culls large parts of the scene using bounding-box tests, so only a few actual intersection tests remain per ray this explains the large speedup.

example8:

Speedup-Faktor = $379.047 / 357.021 = \times 1.06$

Time reduction = $(379.047 - 357.021) / 379.047 = 5.8\%$ schneller

In example8 there are comparatively few primitives (mainly spheres), but a lot of work happens in shading (e.g., area lights with many shadow samples, reflection/refraction). When render time is dominated by many secondary rays and light sampling rather than by a large number of triangle intersection tests, the BVH can only provide limited optimization.